

Topic 2.8

2.8: Smart Objects

In large applications, there may be many different types of objects. It is common to perform similar operations on each object type, such as reading or writing the object. In C, we need different function names for each function that reads or writes an object. This leads to a lot of different functions with very similar, but slightly different, names.

In response to this, programmers began embedding function pointers in their objects. These pointers all had the same name, and pointed to the appropriate function for the object.

This was one practice that lead to the evolution of the C++ language. In C++, the compiler handles many of these details regarding member functions automatically.

The following example in C illustrates this concept:

```
/*
 * FILENAME:    smartobj.c
 * PROGRAMMER:  Gary J. Blair
 * DATE:        01/01/1998
 * LANGUAGE:    ANSI C
 * PURPOSE:     Illustrate the use of smart objects in C.
 */

#include <stdio.h>
#include <string.h>

#define OBJECT_NAME_LENGTH 100
typedef char OBJECT_NAME[OBJECT_NAME_LENGTH];

typedef int (*PRINTF)();
typedef int (*SCANF)();

/* Stuff about a distance. */

typedef struct {
    PRINTF    printf;
    SCANF     scanf;
    OBJECT_NAME name;
    long      feet;
    double    inches;
} DISTANCE;

int printf_distance( DISTANCE *this );
int scanf_distance( DISTANCE *this );

#define INIT_DISTANCE(d,n_feet,n_inches) \
{ \
    d.printf = printf_distance; \
    d.scanf = scanf_distance; \
    strncpy(d.name,#d,OBJECT_NAME_LENGTH-1); \
    d.name[OBJECT_NAME_LENGTH-1] = '\0'; \
    d.feet = n_feet; \
    d.inches = n_inches; \
}
```

```

/* Stuff about a weight. */

typedef struct    {
    PRINTF        printf;
    SCANF         scanf;
    OBJECT_NAME   name;
    long          pounds;
    double        ounces;  }
WEIGHT;

int printf_weight( WEIGHT *this );
int scanf_weight( WEIGHT *this );

#define INIT_WEIGHT(d,n_pounds,n_ounces)    \
{                                             \
    d.printf = printf_weight;               \
    d.scnaf = scanf_weight;                 \
    strncpy(d.name,#d,OBJECT_NAME_LENGTH-1); \
    d.name[OBJECT_NAME_LENGTH-1] = '\0';    \
    d.pounds = n_pounds;                    \
    d.ounces = n_ounces;                    \
}

void somefunc( DISTANCE *dist );
void main(void)
{
    DISTANCE some_dist;
    WEIGHT a_weight;
    INIT_DISTANCE(some_dist,7,5.25)
    INIT_WEIGHT(a_weight,6,4)
    some_dist.printf(&some_dist);
    printf("\n");
    printf("Enter a new value for some_dist: ");
    some_dist.scnaf(&some_dist);
    some_dist.printf(&some_dist);
    printf("\n");
    a_weight.printf(&a_weight);
    printf("\n");
    printf("Enter a new value for a_weight: ");
    a_weight.scnaf(&a_weight);
    a_weight.printf(&a_weight);
    printf("\n");

    somefunc(&some_dist);
}

void somefunc( DISTANCE *dist )
{
    printf("Printing from somefunc ");
    dist->printf(dist);
    printf("\n");
}

int printf_distance( DISTANCE *this )
{
    return printf("%s=%ld'-%lf'",this->name,
                  this->feet,
                  this->inches);
}

int scanf_distance( DISTANCE *this )
{
    return scanf("%ld'-%lf'",&this->feet,
                 &this->inches);
}

int printf_weight( WEIGHT *this )

```

```
{
    return printf("%s=%ld#-%lfoz",this->name,
                  this->pounds,this->ounces);
}

int scanf_weight( WEIGHT *this )
{
    return scanf("%ld#-%lfoz",&this->pounds,
                  &this->ounces);
}
```

Output for smartobj.c:

```
some_dist=7'-5.250000"
Enter a new value for some_dist: 8'-3.5"
some_dist=8'-3.500000"
a_weight=6#-4.000000oz
Enter a new value for a_weight: 12#-6.25oz
a_weight=12#-6.250000oz
Printing from somefunc some_dist=8'-3.500000"
```

Topic 2.9 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-2-dot-9>)